RX / SPORTS ANALYTICS

EPA Implementation Guide

NFL and NBA models
From source observations to market probabilities

LOOKBACK / NEW SEASONS / CODE LOGIC / PRICING

A guide to the implementation in this repository, including the exact history windows, minimum samples, seasonal carryover, chronological calibration, saved forecast behavior and limitations.

Read section 0 first for the answers to how many games are used and what happens when a new season has little or no history.

The original research PDF is retained separately as a source reference. This manual explains what the code actually implements; advanced proposals in the source papers are not represented as completed features.

Code review date: 20 October 2026
Documentation source: Models/Game_Outcome/EPA/README.md

Contents

0. Lookback, new seasons and data readiness 3

0.1 The actual live lookback . . . 3

0.2 New-season behavior, with concrete examples . . . 3

0.3 How old-season weight changes . . . 4

0.4 When we refuse to calculate . . . 4

0.5 Preparing additional history and new-season events . . . 5

0.6 Frozen estimates versus refreshed sources . . . 5

1. Files, reference PDF and commands 6

2. PDF-to-code mapping 7

3. NFL EPA meaning, signs and units 8

4. Source observations and strict coverage checks 8

5. Separate signed passing and rushing rating fits 8

6. EPA advantage, passing share and the fitted scoring bridge 9

6.1 Exclude the league intercept from the advantage . . . 9

6.2 Fit the baseline and conversion together . . . 9

6.3 Pregame volume, mix and score prediction . . . 10

7. EPA uncertainty, probabilities and output objects 10

8. Saved-market service and dashboard 10

9. NBA public-data preparation: what works today 11

10. NBA event-based EPV and pregame forecast 11

10.1 Reconstruct labels before fitting . . . 11

10.2 Fit remaining possession value using information already observed . . . 12

10.3 Signed residual ratings and pregame efficiency . . . 12

10.4 Chronological uncertainty and market settlement . . . 13

10.5 Independent service, archives and criteria . . . 13

11. Validation and differences from the complete PDF system 14

This folder contains an operational **NFL EPA scoring-bridge model** and an operational public **NBA event-based EPV research baseline**. NBA EPA is the dashboard label for the latter's independent spread/total forecast. NCAAF EPA/PPA is not implemented. Shared probability/pricing functions live in the neighboring PPP folder.

Implementation PDF: EPA Implementation Guide. This documents the checked-in NFL EPA and NBA event-based EPV models. The separately named research PDF remains the supplied reference paper.

0. Lookback, new seasons and data readiness

0.1 The actual live lookback

EPA forecasts use **the current sport season and the immediately preceding season**, derived from the saved snapshot cutoff. There is no hard last-four NFL or last-ten NBA rating window. Four and ten are minimum appearances per selected team across all eligible supplied history; they do not mean four or ten games in the current season. All valid earlier requested games may enter the ratings fit. Error calibration has a separate, capped sample.

Rule	NFL EPA	NBA EPA / event-based EPV
Season boundary used by live service	March; January/February belong to the prior NFL season	October; season year is the starting year, e.g. 2026 means 2026-27
Seasons requested	S-1 and S	S-1 and S
Recency half-life	180 calendar days	90 calendar days
Minimum training games	50 distinct league games	100 reconciled distinct league games
Minimum team appearances	4 for each selected team	10 reconciled games for each selected team
Minimum held-out error pairs	40	100
Latest error pairs retained	Up to 250	Up to 300
Holdout grouping	Entire season/week	Entire ISO week
Input availability lag	Game's UTC date +36 hours	Game's UTC date +2 days
Main regularization	Pass/rush team ridge 200; home 20; bridge conversion 25	State ridge 50; team residual ridge 200; home 20

One complete game has two team rows and one paired home/away score error. NBA counts in this table apply **after** scoring/ownership reconciliation. Raw event coverage alone cannot meet the thresholds. NFL EPA also requires complete signed passing/rushing EPA coverage: missing EPA cannot become a zero observation.

0.2 New-season behavior, with concrete examples

For NFL Week 1 in September 2026, the service requests 2025 and 2026. With no completed 2026 games, the 2025 cohort can supply team ratings, the fitted scoring bridge, volume projections and chronological calibration. If the new-season source is unavailable, the data builder can continue with successful earlier data and a source notice. The model does not require four 2026 appearances; four eligible appearances in the combined history suffice for each target team. At Week 3, earlier available 2026 games join rather than replace the old cohort.

For NBA opening night in October 2026, S is 2026 (2026-27), so the requested history is 2025-26 plus 2026-27. Reconciled 2025-26 games can support the EPV state fit, signed team residuals, earlier context/pace projections and error calibration even when there are zero new-season games. Three new-season games can later supplement the old cohort; ten eligible team games are required in total, not ten games in the new season.

The current NBA EPA data builder has an additional cache policy: if at least one requested season already has **both** raw event and possession archives, requested seasons missing either archive are skipped with an explicit notice. It uses the cached seasons rather than repeatedly probing a new season on every fit. This is not a guarantee that a skipped season is unpublished remotely. Once both current-season raw archives are downloaded, that season is considered on the next preparation. If none of the requested seasons is cached, missing public archives are attempted and failed seasons are reported. If no season successfully prepares, the EPA estimate is unavailable.

As a result, do not assume a newer market snapshot automatically fetched the latest NBA event archive. The saved forecast's `source_notices`, `data_quality`, `last_game_date` and input hashes show what was actually used. The raw audit's `nba_epa_ready=false` is an audit-only field, not the gate for the independent reconstructed NBA forecast pipeline.

0.3 How old-season weight changes

Both methods use `w = 2^(-age_days / half_life_days)`. An NFL observation 180 days old has raw weight 0.5; one 360 days old has 0.25. NBA weights at 90 and 180 days are 0.5 and 0.25. Offseason days count, and no fixed season-by-season blend or automatic opening-day reset is used. NFL EPA multiplies recency by pass/rush play counts for its respective rating fits. NBA EPV weights each game's state sufficient statistics by recency and its team residual response by actual possession count times recency.

Unlike NBA PPP's team-prior normalization, NBA EPA does **not** rescale all weights to make the newest observation equal 1. State/team ridge strengths therefore become relatively stronger as all observations age through an offseason. Its administrative-scoring estimate also has a ten-prior-game shrinkage term. `NBAEPAConfig` inherits a `shrinkage_games=10` field from NBA PPP, but that inherited field is not used for EPA team-effect fitting; the administrative-point formula currently uses a literal prior weight of ten. Do not treat that inherited configuration field as an EPA strength slider.

0.4 When we refuse to calculate

Condition	Actual behavior
Zero current-season games, sufficient prior-season history	Fit using eligible prior-season data; disclose unavailable sources.
Too few eligible games for either target team	Return an unavailable-model message; do not substitute league-average or PPP probabilities.
Too few earlier league games or held-out error pairs	No score distribution is returned. More raw rows do not bypass chronology.
NFL signed EPA counts are missing/incomplete	Reject that prepared EPA input instead of treating missing values as zero.
NBA scores, boundaries or ownership cannot reconcile	Quarantine the game and record a reason; no arbitrary reward redistribution.
A whole requested season cannot prepare	Record the source failure and use other successful requested seasons if sufficient.
S-1/S combined history is insufficient	Do not automatically fetch S-2 or lower eligibility minimums.

Minimum training counts alone are insufficient: historical validation needs even earlier training history and both teams' eligible prior appearances. The first part of a dataset often contributes no score-error pairs. Ridge stabilizes a permitted fit but does not override minimum counts or create missing teams. Passing all gates establishes computability, not probability accuracy.

Team identity follows franchise abbreviation. New quarterbacks, coaches, offseason trades, injuries and rotations do not trigger an automatic reset or adjustment. In NBA EPA the model also omits lineup identity, despite raw archives containing lineup fields. These are concrete limitations of cross-season carryover, particularly when a team changes substantially.

0.5 Preparing additional history and new-season events

Use explicit seasons for research or historical cutoffs. NFL CLI defaults follow the computer's current season, not `--as-of`; NBA EPA's standalone CLI currently defaults to `[2025]`. Live services instead select S-1/S from the snapshot cutoff. The following commands download public statistics, not paid market quotes:

```
# NFL: explicitly include an extra historical season for a standalone fit.
python -m Models.Game_Outcome.EPA --seasons 2024 2025 2026 --home BAL --away CIN --as-of
    2026-10-20T03:48:47+00:00

# NBA: explicitly download a new season's missing raw archives and audit them.
python -m Models.Game_Outcome.EPA.prepare_nba_epa --seasons 2026 --download

# Populate independent current-season box-score caches for reconciliation.
python -m sports_analytics.ingest NBA --seasons 2026 --entity team --season-type regular
python -m sports_analytics.ingest NBA --seasons 2026 --entity team --season-type postseason

# NBA: prepare both seasons and fit a standalone forecast at the chosen cutoff.
python -m Models.Game_Outcome.EPA.NBA_EPA --seasons 2025 2026 --home DET --away BOS --as-of
    2026-10-20T03:48:47+00:00
```

Use `--download --refresh` on the raw NBA preparation command to replace existing event archives deliberately. For a new season with no postseason yet, postseason loading may report unavailable; regular-season records can still be used. If the season's event archive is not published or contains no reconciled games yet, the prior season remains the usable source. Expanding a standalone CLI run does not change the live service's fixed two-season request.

NBA EPA reads existing box-score cache entries directly; it does not refresh their age on every fit. Update those public data caches explicitly when needed. The source signature changes when cached box-score provenance or raw file metadata changes, causing prepared training statistics to rebuild. Existing frozen forecast identities are intentionally separate from that rebuild.

0.6 Frozen estimates versus refreshed sources

The live cutoff is the earlier of the saved snapshot timestamp and start time. All inputs must be available strictly before that cutoff and dated before its UTC calendar day. Model version, full configuration, teams, cutoff and neutral status form the cache identity. Source-file hashes are recorded in the artifact, but do not participate in that identity. Therefore a source refresh alone does not automatically replace an already frozen forecast for the same snapshot. Different cutoffs/configurations/versions create new identities. NBA EPA caches are separate from NBA PPP and NFL EPA caches, and an EPA failure never falls back to another model under an EPA heading.

First uncached fits can download free public history; switching models and applying criteria never pull paid odds. Only deliberate Pull Live Markets downloads new quotes. Historical source corrections can change a newly fitted old-date model; time filtering does not guarantee exact historical data vintages.

1. Files, reference PDF and commands

File	Responsibility
EPA.py	Signed NFL passing/rushing EPA ratings, fitted scoring bridge, NFL forecast class and CLI.
NBA_EPA.py	Pre-action NBA EPV fit, signed opponent-adjusted possession residuals, chronological calibration and standalone forecast CLI.
nba_epa_training.py	Score reconstruction, quarantine, pre-action states and per-game sufficient statistics.
nba_epa_data.py	Public NBA event/possession archive downloads, schema checks and score/coverage audits. No NBA forecast is calculated.
prepare_nba_epa.py	NBA input-preparation CLI; offline by default, explicit download/refresh flags.
NBA_EPA_SETUP.md	Setup commands, raw audit findings, reconstruction results and remaining research work.
__init__.py	Public NFL API; `from Models.Game_Outcome.EPA import EPAModel` remains supported.
__main__.py	Package launcher for NFL EPA.

The included EPA research PDF is an unchanged copy of the supplied research note. SHA-256: `f57425deaa9287ec98de598d977e7ead13ceea9050b7c5b7031a7236936998b3`. The body has 15 numbered pages following the cover and contents. References below identify the document's section numbers, not a claim that every proposed extension is implemented.

From the project root:

```
# NFL: prepare observations, fit EPA, and save a standalone forecast.
python -m Models.Game_Outcome.EPA --seasons 2025 2026 --home BAL --away CIN --as-of
    2026-10-19T23:35:23+00:00

# NBA: download missing public archives and audit inputs (not a forecast).
python -m Models.Game_Outcome.EPA.prepare_nba_epa --seasons 2025 --download

# NBA: repeat the audit offline using existing archives.
python -m Models.Game_Outcome.EPA.prepare_nba_epa --seasons 2025

# NBA: prepare reconciled inputs and fit an event-based forecast.
python -m Models.Game_Outcome.EPA.NBA_EPA --seasons 2025 --home DET --away BOS --as-of
    2026-10-20T03:48:47+00:00
```

Direct files also work: `python Models/Game_Outcome/EPA/EPA.py ...` and `python Models/Game_Outcome/EPA/prepare_nba_epa.py ...`. The NFL CLI supports `--refresh` for public historical data and prepares observations alone when no matchup is given. The NBA audit CLI requires `--download --refresh` to replace existing raw archives. The NBA forecast pipeline reuses prepared inputs or downloads missing public archives/box scores, without refreshing existing raw archives. None of these commands calls The Odds API. NBA filenames translate starting-year project seasons to ending-year archive names: project 2025 downloads 2026 assets.

2. PDF-to-code mapping

PDF section	Code	Actual scope
2.1: expected-points game state	Provider nflfastR EPA through `NflModelData`	Uses provider state values/EPA; this repository does not fit a new football EP state model.
2.2: signed reward/possession perspective	`expected_points_added`	Exposes the general reward identity for explanation/tests; the fitted dataset retains the provider's EPA conventions.
2.3: offense/defense EPA allowed	`epa_features` in the NFL data builder	Preserves signed sums/counts, with passing/rushing and offense/defense coverage.
3, 3.1-3.3: team strength, ridge, recency, play mix	`EPAModel._history`, `_fit`, `_means`	Separate signed pass/rush offense/defense effects, centered coefficients, time/sample weights and prior-game projected passing share.
4.1: opportunities	`_means`	Shared drive projection and matchup plays per drive, using earlier observations.
4.2: parsimonious scoring bridge	`scoring_bridge`, `_fit`, `_means`	Fits a nonnegative points-per-drive baseline and EPA conversion coefficient, and adds non-offensive scoring separately.
4.3 and 9: drive-state Monte Carlo	Not implemented	No drive outcome/clock/field-position transition simulator or Monte Carlo betting samples.
5: NCAAF-specific EPA/PPA	Not implemented	NFL EPA is not passed off as college calibration.
6.2-6.3: NBA PPP baseline	NBA_PPP.py	Operational baseline belongs to the PPP folder.
6.1: NBA event-state EPV	`NBA_EPA.py`, `nba_epa_training.py`	Fits expected remaining possession points using coarse public pre-action features; no continuous tracking or shot clock.
6.4: NBA possession simulation	Normal score-error baseline in `NBAEPAModel.predict`	Independent EPV/team means and held-out covariance; possession-by-possession Monte Carlo is not implemented.
7-8: score distribution and examples	`EPAModel.predict`, `EPAForecast`	Independent EPA chronological score errors parameterize the normal margin/total baseline. Examples are formula checks, not live parameters.
10: implied probabilities/fair prices	Inherited forecast/pricing functions in PPP	Push-aware probability and fair American odds, with the app's raw market comparison and Edge.
11-12: engineering and validation	Data manifests, market service, tests and NBA audit report	Cached source provenance and cutoff-aware forecasts exist; full untouched-market validation/production simulation remains further work.

3. NFL EPA meaning, signs and units

An EP value is measured in expected points, relative to a defined scoring horizon and offensive perspective. EPA is the immediate scoring reward plus the next state's signed value minus the prior state's value:

```
EPA = reward + gamma * EP_after - EP_before
gamma = +1 when EP_after uses the original offense's perspective
gamma = -1 when EP_after uses the opponent's perspective
```

`expected_points_added(before, after, reward=0, possession_changed=False)` checks finite inputs and implements that sign switch. The PDF's examples are:

```
Conversion: 4.00 - 2.10 = +1.90 EPA
Interception: -3.00 - 2.10 = -5.10 EPA
```

This helper is not reapplied to all provider records to overwrite nflfastR EPA. The provider's scoring/possession/period conventions remain the source convention. Offensive EPA/play is the offense's signed sum divided by valid play count. Defensive EPA is **EPA allowed**, so a more negative value represents better defense. Taking absolute EPA or replacing missing values with zero would destroy the meaning of those measurements.

4. Source observations and strict coverage checks

NflModelData supplies the same paired team-game records used by NFL PPP: schedules, actual team box scores, nflfastR PBP and PFR snap checks. The PPP documentation explains offensive-play and non-offensive-scoring attribution.

For each side `off`/`def` it additionally stores valid EPA counts and sums, total EPA mean, and passing/rushing counts and sums. `pass` includes spikes, `rush` includes kneels. The two-point tries excluded from the offensive-play denominator are also excluded from this EPA sample. Sacks stay with passing play records.

`EPAModel._history` first checks that the EPA schema exists, then invokes the common history/cutoff checks. It requires:

- All required EPA sums/counts are finite; negative sums are allowed.
- `off_epa_plays == offensive_plays` and `def_epa_plays == defensive_plays`.
- For each side, pass EPA count plus rush EPA count equals total EPA count.
- Both pass and rush counts are positive for each retained team-game row.

Consequently missing EPA cannot quietly become a league-average observation. The `def_*` fields supply coverage/consistency information. Opponent defensive ratings are jointly estimated from offensive response rows facing that opponent; there is not a second independent fit to duplicated defensive response rows.

The NFL source builder conservatively sets availability at game date plus 36 hours. The model uses strictly earlier available observations and earlier UTC game dates. Both sides of each game must be present. Provider source files can contain later historical corrections; they are not exact archival vintages of everything known at an old snapshot.

5. Separate signed passing and rushing rating fits

`EPAModel` inherits the history/validation interface of `PPPModel`, but replaces `_fit` and `_means`. It fits EPA with **weighted squared error**, not the PPP Poisson mean objective. This accepts both positive and negative EPA values.

For play type `k` (pass or rush) and historical team-game `g`:

```
y_gk = off_k_epa_sum / off_k_epa_plays
w_gk = recency_weight_g * off_k_epa_plays

predicted_EPA_gk = alpha_k + offense_k[team_g]
                + defensive_weakness_k[opponent_g] + home_k * h_g
h_g = (2 * is_home - 1) * (1 - neutral) / 2
```

Recency weights use a 180-day half-life. Weighting the game mean by its valid sample count is equivalent to a play-level weighted ridge fit for these predictors, which are constant within a team-game/type cell, up to a constant objective term. It does not create player-level or down/distance context that is absent from X.

With design matrix X, diagonal sample weights W and ridge penalty matrix L:

```
beta_k = solve(X' W X + L, X' W y_k)
```

X contains an intercept, the home indicator, offensive-team indicators and opposing defensive-team indicators. Team coefficients have default ridge penalty 200; the home coefficient has penalty 20; the intercept is unpenalized. Both team-effect sets are centered, absorbing the removed means into the intercept. A positive defensive effect means the opponent allows more EPA and is weaker.

These are declared baseline settings, not hyperparameters established as optimal on untouched future data. Quarterback identity, weather, injuries, rest and starting field position are not regressors in this implementation.

6. EPA advantage, passing share and the fitted scoring bridge

6.1 Exclude the league intercept from the advantage

Let `z_gk = X_g beta_k - alpha_k`. The centered EPA advantage for a prior game is:

```
q_g = off_pass_epa_plays / off_epa_plays
delta_g = q_g * z_g_pass + (1 - q_g) * z_g_rush
```

The intercept is removed deliberately: the scoring bridge's baseline already accounts for league-neutral offensive scoring. The historical training regression uses observed prior-game exposures and passing share. Future-game prediction never substitutes the actual future game's share or play count.

6.2 Fit the baseline and conversion together

PDF section 4.2 motivates the first-stage forecast specification:

```
expected_offensive_points = drives * b + expected_plays * delta * kappa
expected_plays = drives * plays_per_drive
```

Here `b` is baseline offensive points/drive and `kappa` converts the matchup EPA advantage to scoring in this specification. It is a fitted bridge, not a claim that summing EPA is an exact scoreboard identity.

`_fit` constructs two columns `[offensive_drives, offensive_plays * delta]` and targets historical `offensive_points`. It minimizes:

```
sum_g recency_weight_g * (points_g - drives_g*b - plays_g*delta_g*kappa)^2
    + 25 * kappa^2
subject to b >= 0 and kappa >= 0
```

The baseline is unpenalized. The implementation solves the two-variable interior normal equations, considers feasible interior and both boundary fits plus zero, and chooses the smallest penalized objective. Boundary candidates matter because an unconstrained coefficient can be negative. This bridge is fitted on prior training observations; its own training ratings are not cross-fitted within that regression, so untouched-game performance still requires explicit evaluation.

`scoring_bridge` exposes the formula and checks finite inputs and nonnegative drives, length, baseline and conversion. For an illustrative `N=11`, `L=6`, `delta=0.10`, `b=2.0`, `kappa=0.8`, it returns:

```
11*2.0 + 0.8*11*6*0.10 = 27.28 offensive points
```

An additional estimate of 2 non-offensive points would produce 29.28 total points before chronological bias adjustment. These numbers explain the helper; they are not fixed fitted values for any real team.

6.3 Pregame volume, mix and score prediction

`_means` shares a drive projection across the two teams by averaging their weighted offensive and defensive drive counts. Each offense's plays per drive blends its own earlier offense with the opposing earlier defense. Projected passing share similarly blends:

```
q_hat_team = 0.5 * [avg(own off_pass_count) / avg(own off_epa_count)
                  + avg(opponent def_pass_count) / avg(opponent def_epa_count)]
```

For each type, the predicted advantage is own offensive effect plus opposing defensive vulnerability plus the signed home coefficient; neutral sites omit home field. The pass/rush blend supplies `delta_hat`. The offensive bridge result is floored at zero, then additional non-offensive scoring is added using the same two-prior-weight shrinkage as PPP. The neutral `b` is league-level here; there is no projected field-position-specific baseline model.

7. EPA uncertainty, probabilities and output objects

`EPAModel.predict` invokes the common chronological validation driver. Python's method dispatch ensures each held-out week uses **EPA's own** coverage checks, EPA rating fit and scoring bridge. No PPP fitted mean or covariance is reused. The standard defaults are 50 training games, four prior appearances per selected team, at least 40 validation games and the latest 250 validation pairs.

The pair of final-score residuals in each held-out game supplies bias and sample covariance C. The calibrated margin/total parameters are:

```
mu_margin = mu_home - mu_away
mu_total = mu_home + mu_away
variance_margin = C_hh + C_aa - 2*C_ha
variance_total = C_hh + C_aa + 2*C_ha
```

The method returns `EPAForecast`, a subclass of the common forecast class, with EPA-specific diagnostics. It uses the same normal integer-bin settlement logic as PPP. A selected team's handicap `h` wins when its margin plus `h` is positive; integer equality is a refunded push. Over and Under use opposite sides of the same total line.

The displayed probability is `win/(win+loss)`, fair American odds are derived from that conditional probability, and Edge is selected-model probability minus the raw market break-even probability. For probability 0.80 and quote -110, fair odds are -400 and Edge is approximately +27.62 percentage points. The PPP settlement documentation gives exact conversion formulas and a nonzero-push example. The normal sigma values are learned from historical EPA errors; the PDF's worked sigmas are not inserted into live estimates.

8. Saved-market service and dashboard

EPAMarketService reuses the service contract but injects `EPAModel`, `EPAForecast` and method identifier `EPA`. It freezes the earlier of saved snapshot time and kickoff, uses recognized NFL teams, and hashes version/configuration/teams/cutoff/neutral status. Cache files are `data/modeling/nfl/forecasts/epa_<hash>.json`, separate from PPP archives. Standalone CLI exports are readable `epa_<home>_<away>.json` matchup files.

The NFL forecast selector changes probability/fair-odds headings, values, Edge, matching rows and all four comparison charts. Default criteria require both model probability >75% and Edge >5 pp, evaluated across both teams' alternate spreads and both Over/Under totals. Player props have no EPA model probabilities. Unavailable EPA data displays an error with saved prices; it never shows PPP numbers labeled EPA. Selecting models, rendering graphs and applying criteria do not request paid odds. First uncached fits may download public statistics.

9. NBA public-data preparation: what works today

The NBA counterpart of football state valuation is EPV: expected remaining points in a possession given observable information. The event-based public-data approach has been selected; it does not claim the continuous optical-tracking EPV of the paper's richer example.

`nba_epa_data.py` performs the following implemented sequence:

1. `archive_source` translates a project starting year to the ending year in SportsDataverse NBA Stats PBP and derived-possession gzip CSV URLs.
2. `prepare_archive` reuses local files unless an explicit download/refresh is requested. Fresh downloads must have required columns and pass a complete gzip read before atomic replacement. Manifests record source URL, retrieval timestamp, season translation and SHA-256. No fabricated data fills gaps.
3. `archive_columns` checks required possession ownership, counts/points, identifiers, event timing/order and scoreboard column names. This verifies schema presence, not all possession-boundary semantics.
4. `audit_archives` streams chunks instead of loading whole seasons into RAM. IDs beginning `002`/`004` select regular/postseason games. Other prefixes, including preseason, All-Star and play-in, are counted as outside the scope.
5. The audit counts coverage, duplicate possession keys, invalid point/count/ ownership records and populated unique ten-player lineup IDs. It retains points from non-counted records instead of confusing scoring with a possession denominator. Populated IDs are not proof of correct lineup reconstruction.
6. Possession points are aggregated by game/offense team and compared to cached team box scores. Separately, the last event by period/order supplies home/away final scores for a second comparison. Missing rows and mismatches retain IDs and values in the report; there is no automatic discrepancy redistribution.
7. `prepare_inputs` combines those results in `data/modeling/nba/epa/readiness.json`, preserving raw archives separately. Existing box-score cache entries are read without requesting additional data.

The initial 2025-26 audit found 1,315 games, 665,170 events, 313 possession-score discrepancies in 2,630 team-game comparisons, 13 invalid possession records and two final-event-score discrepancies. These are a recorded source/audit snapshot, not immutable claims about future refreshed archives. The report is the current source of truth. The inspected files lack shot-clock readings and continuous tracking coordinates. Event shot locations must not be relabeled as tracking.

`nba_epa_ready` is **always false in this raw-audit code**. Even zero raw discrepancies would not mean that a model has been trained or validated. The CLI can successfully prepare an audit report containing data-quality failures; its success status means preparation completed. This field is not the dashboard gate for the separate reconciled forecasting pipeline described below.

10. NBA event-based EPV and pregame forecast

10.1 Reconstruct labels before fitting

`NBAEPAData.prepare` loads regular/postseason box scores and public event and possession archives. `reconcile_game` requires two recognized teams, unique ordered events, non-overlapping possession intervals and matching period/ownership. Scoreboard fields are carried forward from an explicit 0-0 start. Final event scores must equal independent box scores. Incremental scoring must be nonnegative, integer, at most four points per action and credited to only one team.

Provider possession `points` is not a trusted target. The code reconstructs it from scoreboard increments within each interval. Identified technical/defensive three-second scoring is separated as administrative points; other scoring must belong to the possession's offense. Non-counted intervals may not hide ordinary scoring. Actual counted possessions form offensive/defensive denominators, and ordinary plus administrative points must reproduce both final scores exactly. Ambiguous score corrections, opposing-offense rewards, overlapping boundaries or final-score gaps

quarantine the entire game with a reason. No discrepancy is spread arbitrarily across possessions.

For the inspected 2025-26 files, this retains **1,175 of 1,315 games**, reconstructs **582 possession-point labels**, and supplies **587,738 pre-action states**. The 140 excluded games comprise 95 ambiguous scoring increments/corrections, 44 opposing-possession scoring cases and one final-score mismatch. These counts describe the local source revision; refreshed archives can differ. Exclusions can produce a nonrandom sample and should be evaluated as a modeling limitation.

Each prepared season writes a team-game CSV, source/signature manifest, `state_statistics_<season>.npz`, a state sample CSV and quarantine JSON under `data/modeling/nba/epa/`. Versioned signatures include raw file metadata and cached box-score provenance. Raw files remain unchanged. The NPZ stores per-game X'X and X'y with no pickled objects; these are lossless sufficient statistics for the declared linear weighted state fit, not for arbitrary nonlinear models.

10.2 Fit remaining possession value using information already observed

The label immediately before each action is the ordinary points still to be scored by that same offense in that possession. `state_features` has 17 columns: intercept; normalized game clock; period 2/3/4 and overtime indicators; elapsed possession seconds /24; offensive rebounds so far; points so far /3; four possession-start context indicators (made shot, missed shot, live-ball turnover, timeout); and four previous-action indicators (shot, rebound, turnover, free throw). Regulation clock divides by 720 and overtime clock by 300. These are coarse event features, not a continuous tracking representation.

Current-action reward/outcome is accumulated **after** its feature row is built. An offensive rebound becomes context for subsequent actions and does not reset the offensive horizon. Terminal remaining value is zero. The next offense's value is never used as continuation of the original offense. Current shot outcome/location, actual future lineups and target-game pace are not inputs.

For earlier eligible game g, `w_g = 2^(-age_days/90)`. The state ridge fit is:

```
beta = solve(sum_g w_g * X_g'X_g + L, sum_g w_g * X_g'y_g)
V(state) approximately equals X_state * beta
L = diag(0, 50, ..., 50)
```

The intercept is unpenalized. `expected_possession_value` exposes a nonnegative state estimate; training and prediction require mean starting-possession values to be finite and positive. The fitted state model remains a linear approximation, not a calibrated conditional distribution for every individual event.

10.3 Signed residual ratings and pregame efficiency

Under this same-offense convention, action value is `reward + V(next_same_offense_state) - V(current_state)`. Its sum telescopes to `possession_points - V(start)`. The model therefore fits team effects to the mean signed possession residual directly:

```
y_team_game = ordinary_points / actual_offensive_possessions - mean_start_V
y_hat = alpha + offense[team] + defensive_weakness[opponent] + home * h
h = +0.5 for home, -0.5 for away
weight = actual_offensive_possessions * recency_weight
```

This squared-error ridge preserves negative values. Team penalties are 200, home penalty 20, intercept penalty zero; offense and defense effects are centered. Positive defensive weakness means more expected points allowed. There is no second fit to duplicated defensive responses. The state and residual stages fit the same earlier training cohort; they are not internally cross-fitted. Whole held-out weeks remain outside both fits.

For each target team, projected starting features average its earlier offensive contexts and the opponent's earlier defensive contexts, with recency weighting. Starting EPV plus fitted signed residual effects yields expected ordinary points/possession. A shared possession forecast averages both teams' earlier offensive and defensive actual counts. Technical/administrative points are added separately, shrinking each team's weighted average toward the league average with ten prior-game weights:

```
expected_score = shared_possessions * (projected_start_V + signed_advantage)
               + expected_administrative_points
```

Neutral venues remove the home effect. This replaces NBA PPP's fitted mean; it does not add EPA a second time to an already predicted PPP score.

10.4 Chronological uncertainty and market settlement

The shared NBA history validator requires complete paired games and strictly earlier game/availability timestamps. Each ISO week is held out while both EPV stages use only games available before that week's earliest game. Defaults are 100 training games, ten prior games per team, at least 100 held-out score pairs and the latest 300 for calibration. The final target fit uses all eligible history.

Held-out home/away score errors provide bias and covariance C. Margin variance is `C_hh+C_aa-2*C_ha`; total variance is `C_hh+C_aa+2*C_ha`. Nonpositive variance is rejected. These normal marginals are the declared baseline alternative to the PDF's possession Monte Carlo. Historical final scores include overtime, but no explicit overtime simulator is fitted. NBA spread settlement removes the zero-margin bin and renormalizes, because final games cannot tie; other integer handicap/total pushes are retained. As in PPP, displayed probability is `win/(win+loss)`, fair odds follow that probability, and Edge subtracts the raw market break-even probability. This is not a fully coherent discrete joint score simulation conditioned on every basketball scoring rule.

10.5 Independent service, archives and criteria

`NBAEPAMarketService` uses method EPA and its own model version (currently 2), configuration and forecast class. It freezes the earlier of saved quote time and tipoff. Hashed artifacts are written under `data/modeling/nba/epa/forecasts/`, including source notices and SHA-256 hashes of prepared team games, manifests and state statistics. Existing archives restore without a fit or source request. If a requested current-season archive is unavailable but earlier cached history exists, that history can be used with an explicit source notice. Missing or invalid inputs show EPA unavailable; they never substitute NBA PPP or NFL values.

NBA's forecast dropdown retains PPP by default and enables EPA. Selection changes probability/fair-odds headings and values, Edge, highlights and all four charts. Apply Criteria opens Game Lines and evaluates both teams' alternate spreads and both Over/Under totals. Both model probability >75% and Edge >5 pp are required by default. A status summary reports counts per group, zero matches or missing estimates. Extra selections stay collapsed until Show more is clicked; chart segments and markers for qualifying lines appear in gold. Switching models or applying criteria never requests paid odds. Player props remain prices only.

The research baseline omits tracking, shot clock, player/injury forecasts and an explicit possession simulator. Historical providers can revise old events; time filtering does not recreate exact historical source vintages. Chronological score-error calibration is not untouched-future probability validation. Reserve additional seasons for Brier/log-loss/reliability/tail evaluation and assess the effects of quarantined-game selection before relying on the probabilities.

11. Validation and differences from the complete PDF system

Run `python -m unittest discover -s tests -q`. NFL EPA tests cover signed reward examples, bridge arithmetic, missing EPA rejection, cutoff filtering, covariance, cache separation and method-dependent UI/Edge behavior. NBA preparation tests cover year translation, no implicit downloads, administrative scoring, preseason exclusion, schema changes, duplicated keys and independently reported score gaps. NBA model tests additionally cover score reconstruction, technical scoring, pre-action features, rejection of inconsistent games, future-data poisoning, normal/push complements, independent frozen EPA restoration and method-dependent NBA UI/criteria behavior. All tests are offline and consume no paid API credits.

The implemented NFL bridge/normal baseline omits the PDF's preferred drive-state Monte Carlo, explicit kickoff/field-position transitions, QB/weather/injury features, full football overtime transitions, NCAAF calibration and staking/ cross-book execution. Historical final scores include actual overtime, but that is not an explicit overtime rules engine. Normal push/key-number approximations remain limitations. NBA EPV training is implemented as the event-based baseline above; richer tracking models and possession simulation remain further work. Engineering tests and calibration fits do not establish forecasting profitability or untouched-future probability calibration.

Build provenance

The following SHA-256 hashes identify the reviewed calculation/service files. They are documentation provenance, not a runtime cache identity.

Models/Game_Outcome/EPA/README.md
`98b7f543fb5e50ef8804e99cb3751da2ec3444eaefc67ec249a093f45d71ef86`

Models/Game_Outcome/EPA/EPA.py
`15cee5b5244bb87a360a9d2035973877eedc82cfe86798cf28cd5d1fe28c7890`

Models/Game_Outcome/EPA/NBA_EPA.py
`42a7df562be8313cc0bc099a164afae3ba53c97104e06467bfa5439025d087f9`

Models/Game_Outcome/EPA/nba_epa_training.py
`4ee17093053429c5080d20a55adc339661445e95a537927501feef35b8e744ac`

Models/Game_Outcome/EPA/nba_epa_data.py
`5885a9afed702d46c864201344efa5cf50f8118a1c7112b9cf942d5aa408e7b2`

sports_analytics/epa_service.py
`00a05ab1d1d1d89b3f5002ae4f92bfa6c6aeb8560d3f60375af8b77cbaac32af`

sports_analytics/nba_epa_service.py
`07dc860d63ab57c8c0348a05d0ccc915781aeb873686b0513e09fa7228931dbc`

sports_analytics/ppp_service.py
`f48d098f2ab4ff440acc748e214e191a753f6d0c03b9d2f31db202bcdd984f68`